# AJ_24038155_dialect_v1.4_to_v1.5 update

## Change specification from AJ_24038155 Dialect v1.4 to v1.5

### Status

This document records the v1.4 to v1.5 update to the frozen AJ_24038155 Dialect v1.4 specification. All v1.0, v1.1, v1.2, v1.3 and v1.4 lexical, syntactic and semantic rules remain normative unless explicitly changed below.

Version v1.5 is introduced because the baseline corpus contains explicit source comments such as `COMMENT import string` together with a source-level standard-library adapter spelling, `string_method_punctuation`. Under v1.4 every COMMENT is non-executable, so the declared module dependency cannot be represented faithfully and the adapter remains unresolved.

v1.5 adds an exact import-directive family and one explicit punctuation adapter. It does not make general comments executable and does not infer unspecified library semantics.

## 1. COMMENT import directives

### 1.1 Canonical syntax

```
COMMENT import <module>
COMMENT import <module> AS <alias>
COMMENT from <module> import <name>
COMMENT from <module> import <name1>, <name2>, ...
COMMENT from <module> import *
```

Examples:

```
COMMENT import string
COMMENT import numpy AS np
COMMENT from math import sqrt
COMMENT from math import sqrt, floor
COMMENT from math import *
```

The wildcard form follows Python ordering: `from <module> import *`. `from * import <module>` is not valid AJ v1.5 syntax.

### 1.2 Grammar extension

```
comment ::=
    ordinary_comment
  | import_directive ;

ordinary_comment ::=
    "COMMENT" , comment_text , newline ;
```

```
import_directive ::=
    "COMMENT" , "import" , module_reference , [ "AS" , identifier ] , newline
  | "COMMENT" , "from" , module_reference , "import" , import_target_list , newline ;

module_reference ::=
    identifier , { "." , identifier } ;

import_target_list ::=
    "*"
  | identifier , { "," , identifier } ;
```

### 1.3 Recognition rule

Only a complete COMMENT payload matching one of the grammar forms is executable.

Therefore:

```
COMMENT import string
```

is an import directive, while:

```
COMMENT import string later if needed
COMMENT this algorithm would import string in Python
COMMENT from * import math
```

remain ordinary comments.

### 1.4 Python translation

```
COMMENT import string
-> import string

COMMENT import numpy AS np
-> import numpy as np

COMMENT from math import sqrt, floor
-> from math import sqrt, floor

COMMENT from math import *
-> from math import *
```

The compiler emits the import at the corresponding lexical position. It does not hoist, add or infer imports that the source did not declare. Wildcard imports are restricted to program/module scope.

### 1.5 AST and flow representation

The parser emits a dedicated `ImportDirectiveNode` for recognised directives. Ordinary comments continue to emit `CommentNode`.

`ImportDirectiveNode` records the directive type, module, optional alias and imported-name list. The flow model classifies it as `IMPORT` so that executable dependency declarations remain visible in structural evidence rather than being collapsed into comments.

## 2. Standard-library punctuation adapter

### 2.1 Canonical adapter

The exact AJ identifier:

```
string_method_punctuation
```

has the deterministic Python lowering:

```
string_method_punctuation
-> string.punctuation
```

The source must still declare `string` where required:

```
COMMENT import string
```

### 2.2 Example

AJ v1.5:

```
COMMENT import string

IF char NOT IN string_method_punctuation THEN
    APPEND char TO filtered_chars
ENDIF
```

Python target:

```
import string

if char not in string.punctuation:
    filtered_chars.append(char)
```

### 2.3 No general punctuation inference

This addition does not define the meanings of arbitrary punctuation predicates or phrases. In particular, undefined forms such as `is punctuation`, `ispunct`, or a bare prose concept `punctuation` remain outside the dialect unless separately specified by a later version.

## 3. Static semantic refinement

A direct module import binds the module's root name, or its explicit alias when `AS` is used.

A `from` import binds each explicitly listed imported name. Wildcard-imported names are not statically enumerated by AJ validation.

No import is inserted implicitly to satisfy an unresolved identifier.

## 4. Baseline Algorithms 5 and 7

The frozen baseline pseudocode for Algorithms 5 and 7 contains:

```
COMMENT import string
```

and tests characters using:

```
char NOT IN string_method_punctuation
```

Under v1.5 those source-authored constructs are represented directly as an executable import directive plus the explicit standard-library adapter. This is a language-version change, not an algorithm-specific compiler exception.

## 5. Backwards compatibility

All canonical AJ v1.4 programs remain valid under v1.5.

Ordinary COMMENT text retains its v1.4 non-executable meaning unless the complete payload exactly matches a v1.5 import directive.

`CALL ISINSTANCE(...)`, explicit runtime casts, `QUIT`, native `NONE`, dictionary/member-call rules and all previous control-flow semantics remain unchanged.

## 6. Version identification

This specification delta is frozen as:

**Cambridge-Style Pseudocode - AJ_24038155 Dialect v1.5**

Any later syntax addition, removal or semantic alteration requires a new dialect version.